

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1

Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

3. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

4. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

5. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output

Dataset summary and statistical information.

Task 2: Exploratory Data Analysis

Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

6.Source Code

https://github.com/SamrithaShree/Deep_Learning/tree/main/Lab-1

7. Experimental Results

First Five Samples

Index	Variance	Skewness	Curtosis	Entropy	Class
0	3.62160	8.6661	-2.8073	-0.44699	0
1	4.54590	8.1674	-2.4586	-1.46210	0
2	3.86600	-2.6383	1.9242	0.10645	0
3	3.45660	9.5228	-4.0112	-3.59440	0
4	0.32924	-4.4552	4.5718	-0.98880	0

Dataset Dimensions

- Number of Instances: **1372**
- Number of Features (including target): **5**
- Dataset Shape: **(1372, 5)**

Missing Values

No missing values were found in the dataset.

Feature	Missing Values
Variance	0
Skewness	0
Curtosis	0
Entropy	0
Class	0

Descriptive Statistics

Statistic	Variance	Skewness	Curtosis	Entropy	Class
Count	1372.000000	1372.000000	1372.000000	1372.000000	1372.000000
Mean	0.433735	1.922353	1.397627	-1.191657	0.444606
Std Dev	2.842763	5.869047	4.310030	2.101013	0.497103
Minimum	-7.042100	-13.773100	-5.286100	-8.548200	0.000000
25%	-1.773000	-1.708200	-1.574975	-2.413450	0.000000
50% (Median)	0.496180	2.319650	0.616630	-0.586650	0.000000
75%	2.821475	6.814625	3.179250	0.394810	1.000000
Maximum	6.824800	12.951600	17.927400	2.449500	1.000000

Dataset Information

- Total Entries: **1372**
- Total Columns: **5**

- Numerical Features: **4**
- Target Column: **Class**
- Data Types: **4 float64, 1 int64**
- Memory Usage: **53.7 KB**

Inference: The dataset consists of 1372 samples without any missing values, indicating it is optimal for conducting analysis. Training models on data without cleaning. All of the input characteristics are numerical and the descrip- The statistics of various features show different distributions. Therefore Normalizing The Feature is an important task. Preparation before the training of the perceptron.

Data Preprocessing Summary

The training and testing sets used were normalized by the MinMaxScaler of Python library.
A ratio of 80:20

- Training Features: **(1097, 4)**
- Testing Features: **(275, 4)**
- Training Labels: **(1097,)**
- Testing Labels: **(275,)**

8. Generated Plots with Interpretation

1. Feature Histograms

Purpose: Distribution Analysis

Interpretation: The diagrams show the allocation of each feature for fake and genuine. Fake Currency Two classes corresponding distributions of most features are slightly skewed. They possess some degree of uniqueness in distribution and some extreme values, which indicates they are useful for classification.

2. Correlation Heatmap

Purpose: Feature Relationship

Interpretation: The correlation heatmap shows how input features relate to each other. Some features show moderate positive or negative correlation no pair is perfectly correlated. showing that all features contribute useful information.

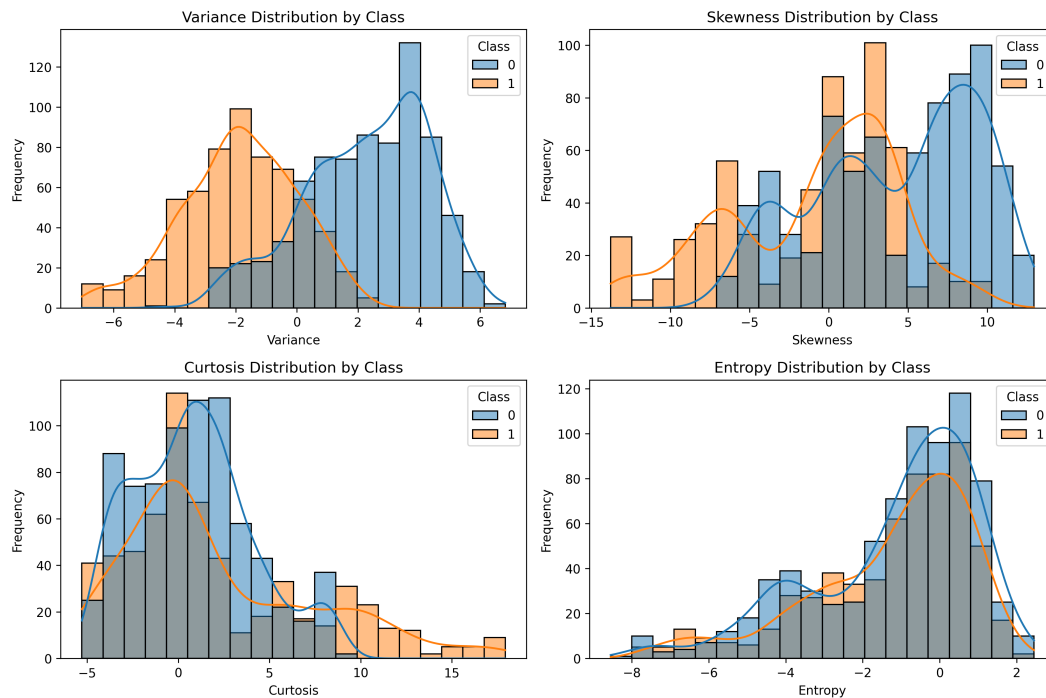


Figure 1: Feature Histograms for Banknote Authentication Dataset

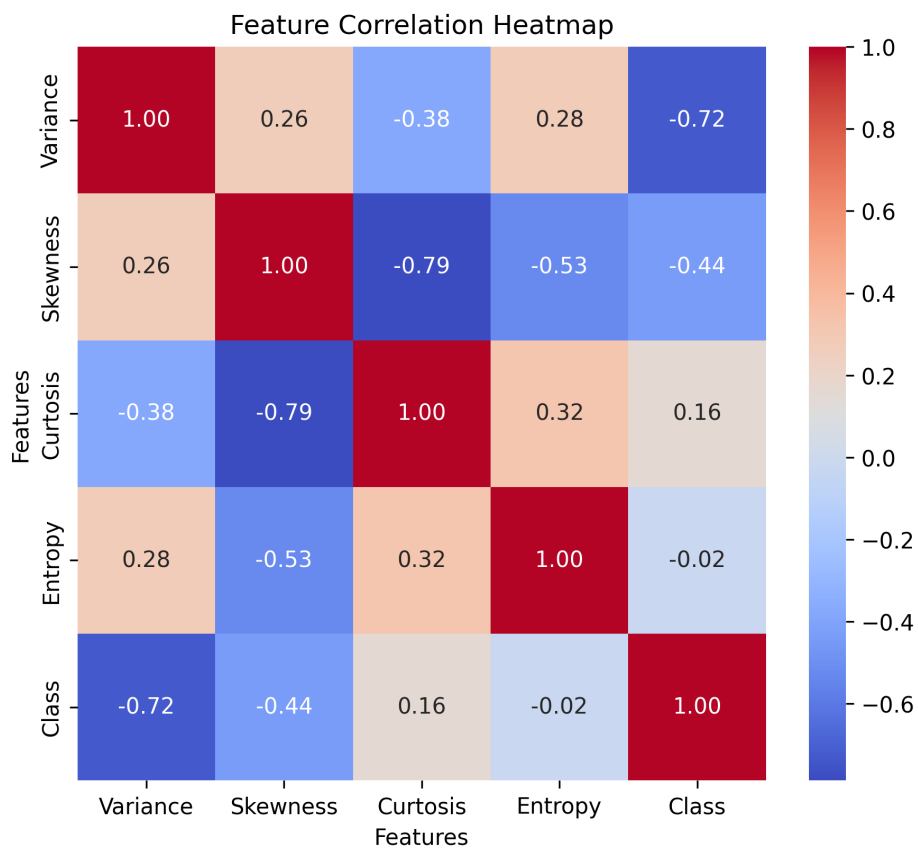


Figure 2: Correlation Heatmap

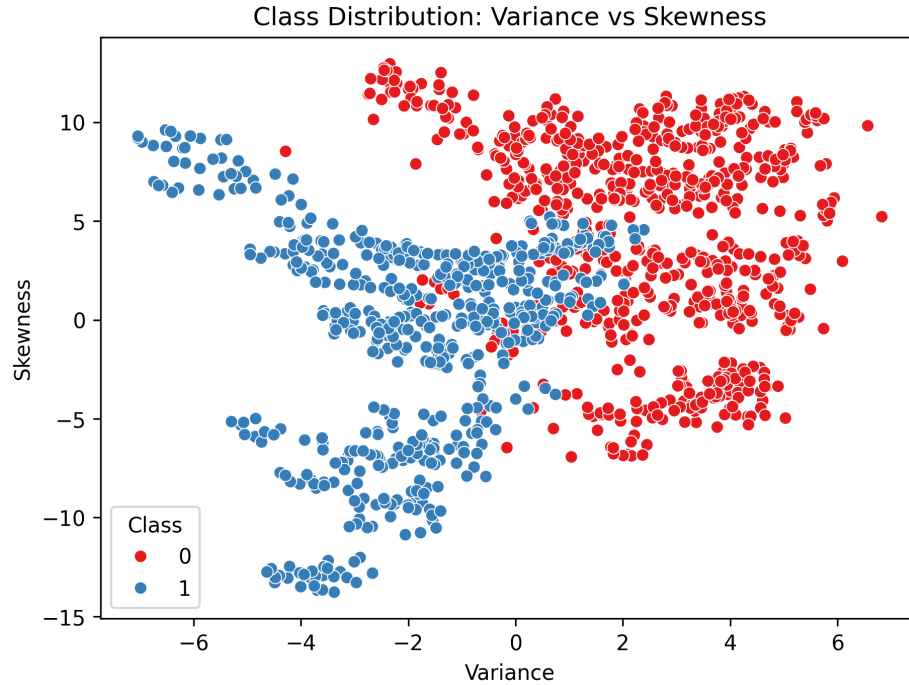


Figure 3: Scatter Plot of Variance vs Skewness

3. Scatter Plot

Purpose: Class Distribution

Interpretation: Based on the scatter plot, we can interpret that bona fide and counterfeit banknotes are largely different. The feature space that has been selected, is shown as formable. A linear classifier like the perceptron can achieve this. Most samples can be classified efficiently.

4. Feature Boxplots

Purpose: Distribution Analysis

Interpretation: The boxplots give us an idea about the spread of each feature. Due to the differences in values among the features normalization is needed. Trainings.

5. Training Error vs Epoch

Purpose: Learning Behaviour

Interpretation: According to the analysis, we see that fewer and fewer samples are misclassified with respect to the number of epochs. . . stabilizes ultimately. This means that the perceptron manages to find a suitable decision. Limit.

6. Weight Evolution

Purpose: Parameter Analysis

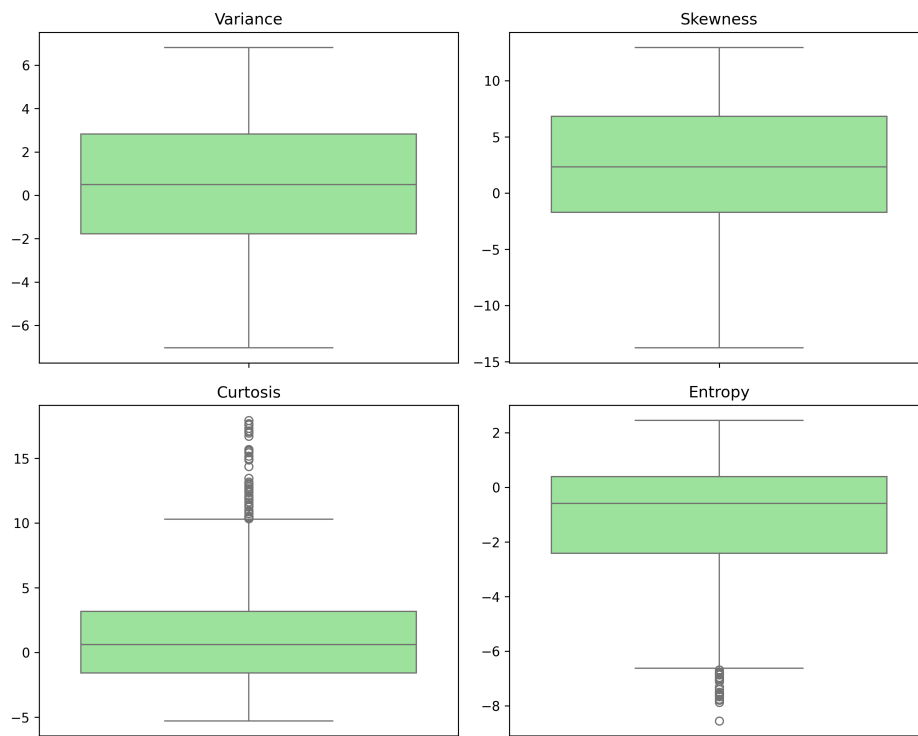


Figure 4: Feature Boxplots

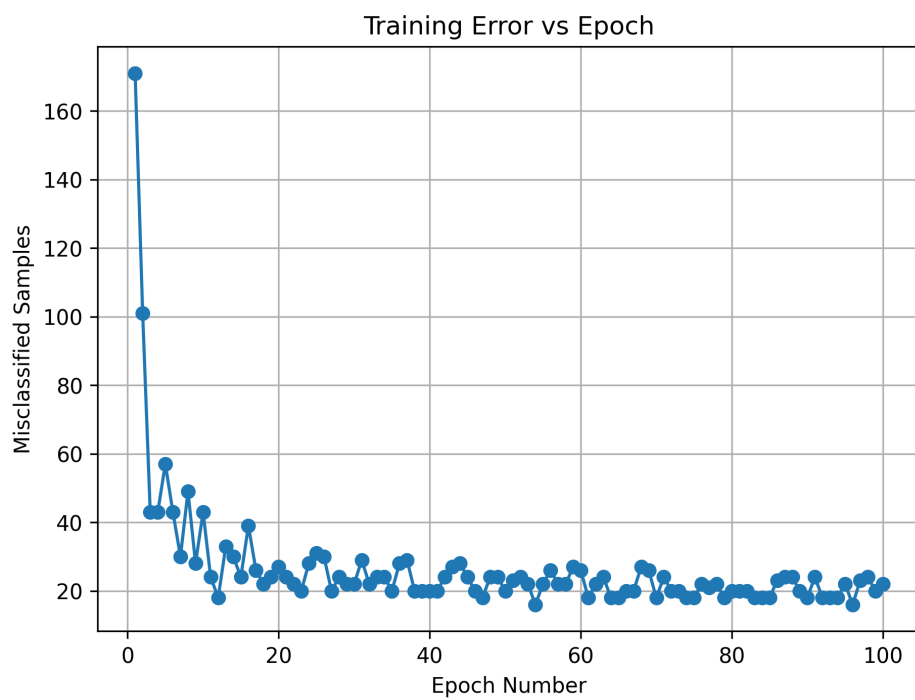


Figure 5: Training Error vs Epoch

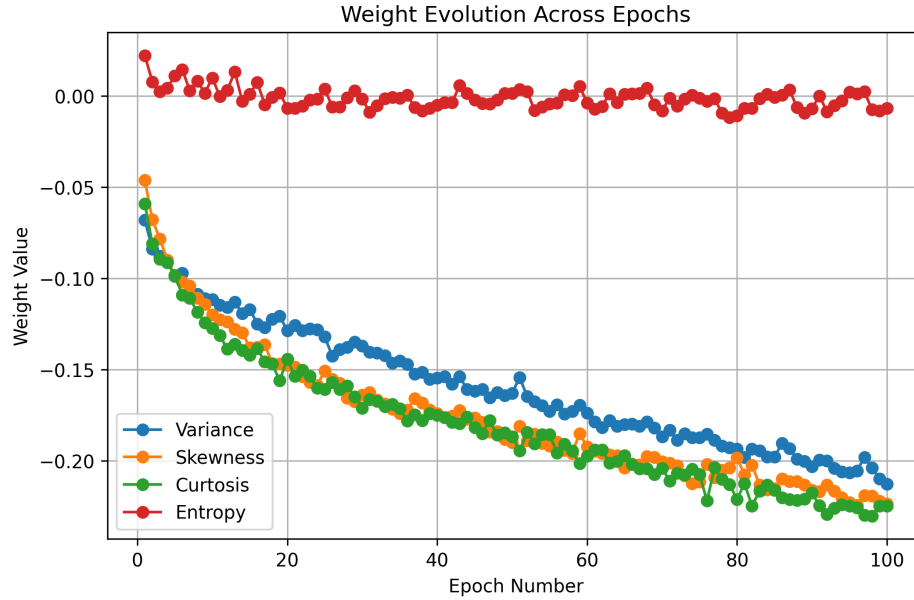


Figure 6: Weight Evolution Across Epochs

Interpretation: In the interpretative dimension, the feature weights change suddenly in initial epochs and follow through a gradual adjustment. Values of a parameter improve with training progress. It can be inferred that the learning algorithm is optimal.

7. Bias Evolution

Purpose: Parameter Analysis

Interpretation: The bias value during training changes to adjust the decision boundary and. It condusively converges to about 0.29.

8. Confusion Matrix

Purpose: Performance Evaluation

Interpretation: The confusion matrix shows that the perceptron almost classified correctly all test samples, with a misclassification rate of only 3. It has high True Positive (TP) and True TN values that are negative with very few FP and FN resulting in an accuracy of 98.91

9. Learning Rate Comparison

Purpose: Hyperparameter Study

Interpretation: The comparison of learning rate shows that larger learning rate converges faster smaller learning rate converges slower a learning rate of 0.01 leads to a good tradeoff between convergence speed and stability for this experiment.

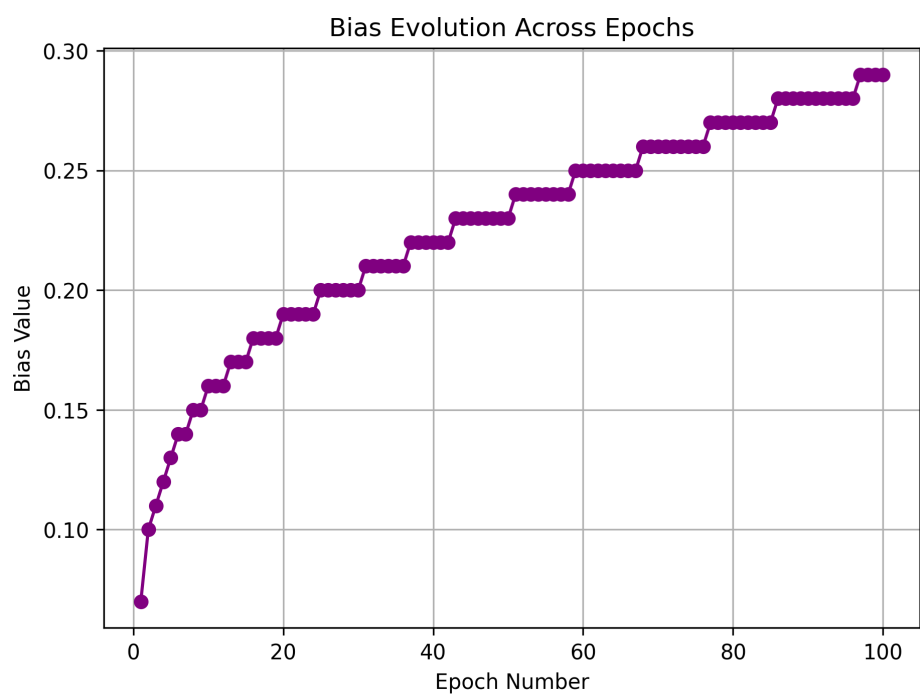


Figure 7: Bias Evolution Across Epochs

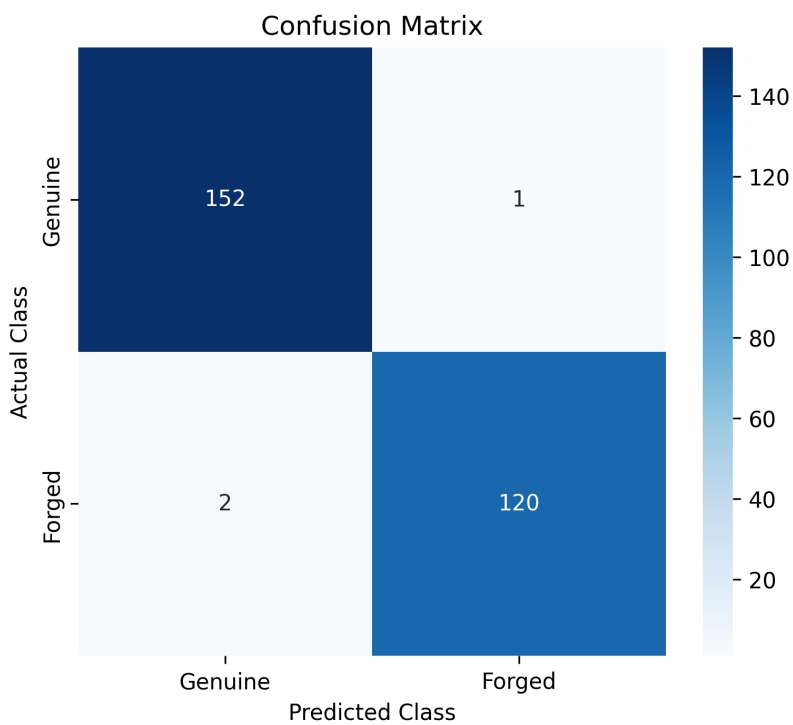


Figure 8: Confusion Matrix

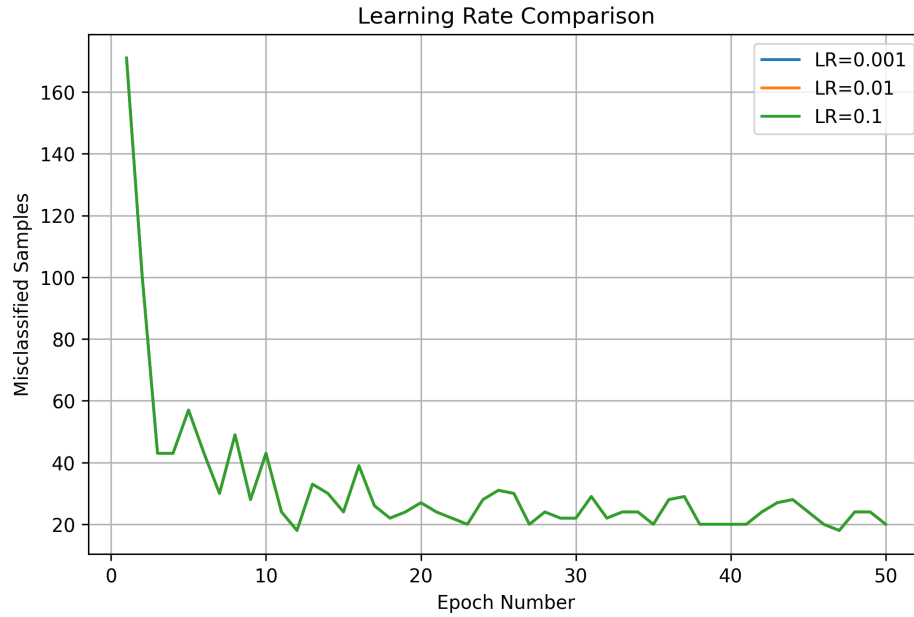


Figure 9: Learning Rate Comparison

9. Performance Tables

Training Summary

Dataset Size	1372
Train/Test Split	1097 / 275
Learning Rate	0.01
Epochs	100
Final Weights	[-0.212646,-0.223446,-0.224733,-0.006686]
Final Bias	0.29
Accuracy	98.91
Precision	99.17
Recall	98.36
F1-score	98.77

Epoch-wise Learning

Epoch	Errors	Weight 1	Weight 2	Bias
1	171	-0.067924	-0.046173	0.07
25	31	-0.131923	-0.150585	0.20
50	20	-0.162813	-0.189793	0.23
100	22	-0.212646	-0.223446	0.29

10. Additional Tasks

1. Comparison of Step and Sigmoid Activation Functions

Feature	Step Function	Sigmoid Function
Output Range	0 or 1	0 to 1
Nature	Discontinuous	Continuous and smooth
Differentiable	No	Yes
Gradient-based Learning	Not Supported	Supported
Applications	Classical Perceptron	Modern Neural Networks

Discussion: The Step Activation Function is not used in modern deep learning because it is non-differentiable almost everywhere and zero gradient, making backpropagation impossible. The Sigmoid function is differentiable and hence suitable for gradient based optimization.

2. Comparison with Scikit-learn's Perceptron

The custom implementation manually implements the original perceptron learning rule by weight initialization, forward propagation, step activation and weight updates. Scikit-learn's implementation is optimized, converges more quickly, supports regularization, early stopping, and a few optimization decisions. Both implementations conduct binary linear-classification, but Scikit-learn is more efficient and practical for the applications.

3. Effect of Different Learning Rates

The perceptron was trained with learning rates 0.001, 0.01 and 0.1.

- **0.001:** Slow convergence, need more epochs.
- **0.01:** Stable convergence, good classification results.
- **0.1:** Faster learning but larger weight updates lead to fluctuations before convergence.

Of the three values, a learning rate of 0.01 yielded the best compromise between convergence speed and the stability.

4. Why a Single Layer Perceptron Cannot Solve the XOR Problem

The XOR problem is not linearly separable, meaning that a single straight decision boundary cannot correctly classify all 4 input combinations. Since Single Layer Perceptron can learn only. It cannot solve XOR problem, as it has linear decision boundaries. A Multilayer Perceptron (MLP) non-linear relationship with one or more hidden layers to learn.

5. Effect of Feature Normalization on Model Convergence

Feature normalization is used to scale all the input features to a common range. This prevents features with larger values from dominating the learning process. In this experiment Min-Max normalization resulted in faster convergence, more stable weight updations and better classification performance as compared to un-normalized data.

6. Perceptron Learning Behaviour for Logic Gates

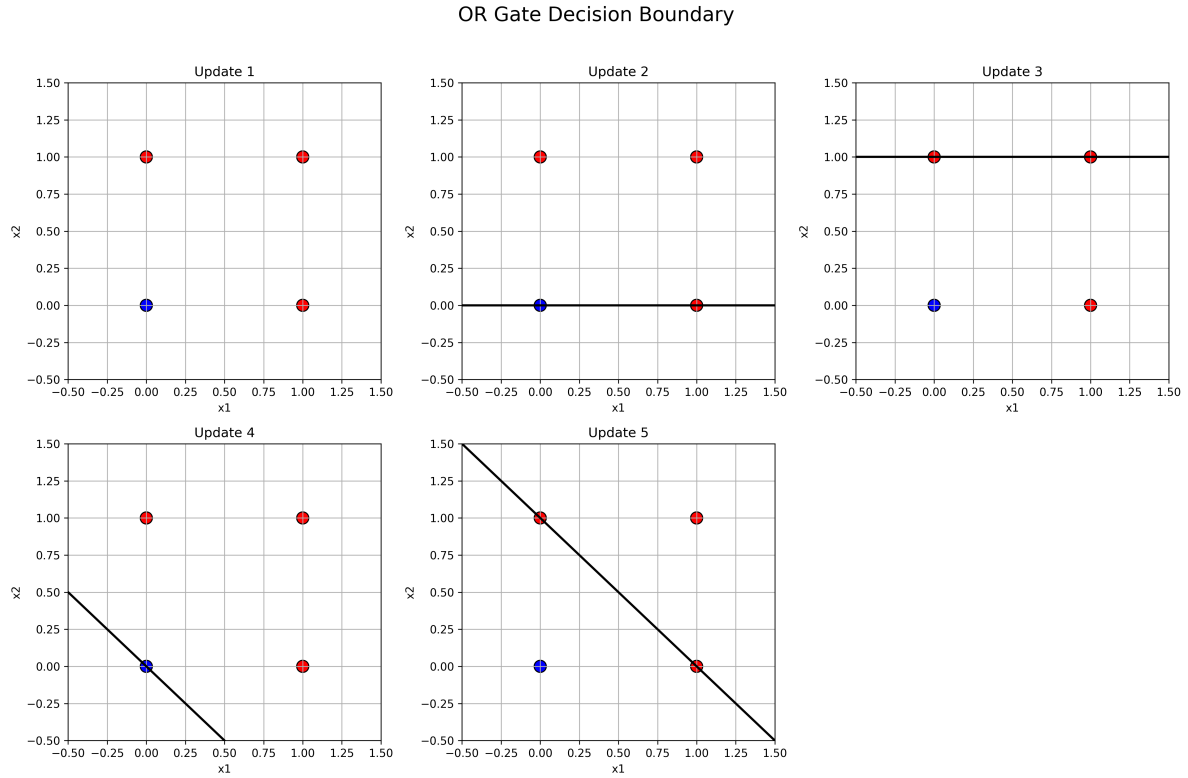


Figure 10: Evolution of the Decision Boundary During Perceptron Learning for the OR Gate

AND Gate Decision Boundary

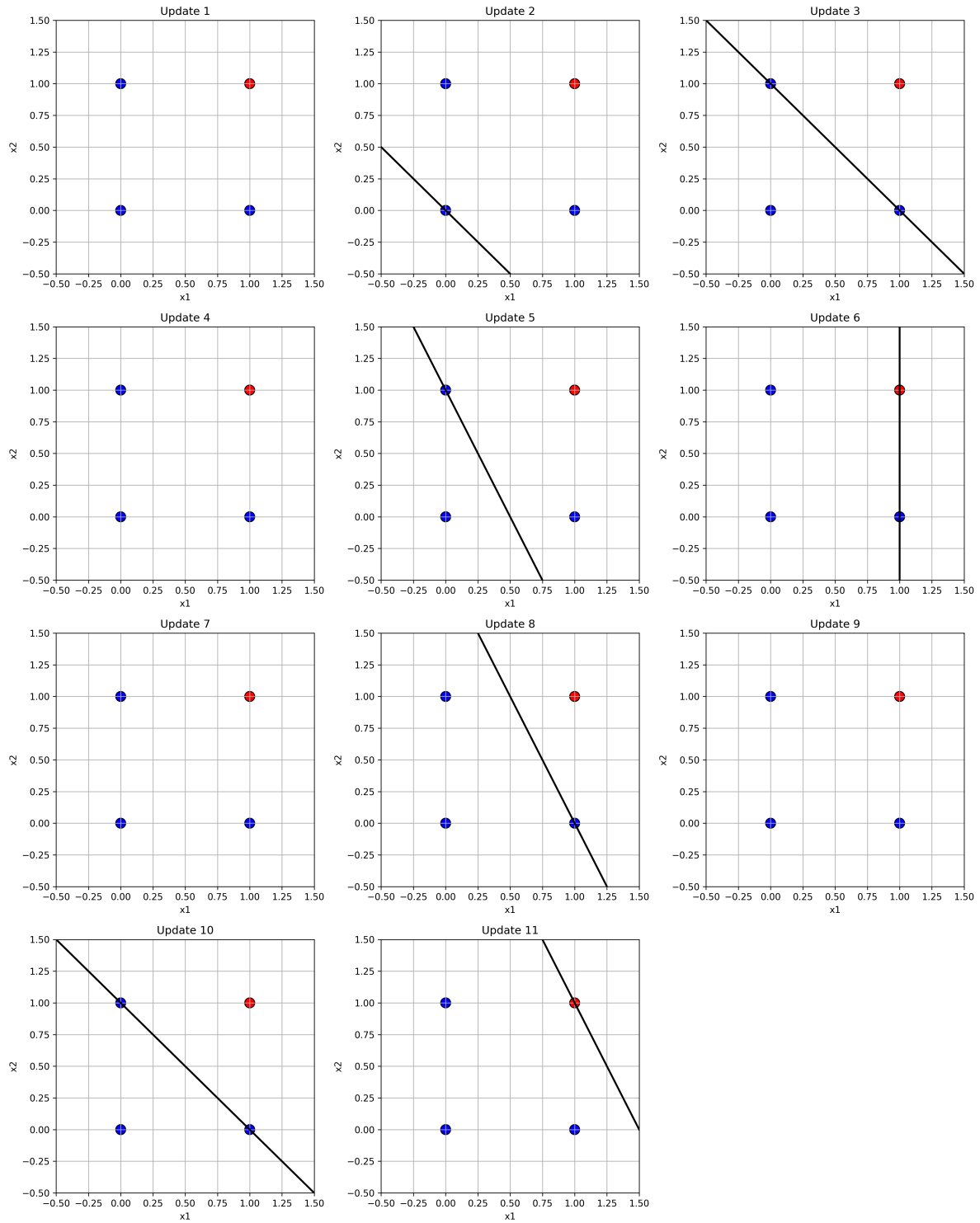


Figure 11: Evolution of the Decision Boundary During Perceptron Learning for the AND Gate

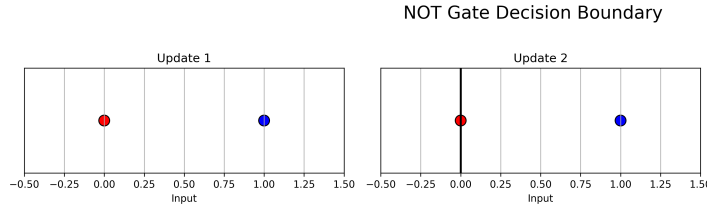


Figure 12: Evolution of the Decision Boundary During Perceptron Learning for the NOT Gate

10. References

1. F. Rosenblatt, “The Perceptron,” *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.